



Documentation Technique

Application RAG Multi-Agent

Analyse de Projet

Documents Départementaux sur les Risques Majeurs

Date: 21 Janvier 2026

Version: 1.0

Table des matières

[Vue d'ensemble](#)
[Architecture Système](#)
[Analyse Détaillée des Composants](#)
[Flux de Traitement](#)
[Dépendances et Intégrations](#)
[Résultats Attendus](#)
[Configuration et Déploiement](#)
[Guide de Maintenance](#)

À propos de ce document

Cette documentation technique est un guide complet pour comprendre, maintenir et évoluer l'application RAG Multi-Agent. Elle couvre l'architecture globale, l'analyse détaillée de chaque composant, les flux de traitement, et les guide de maintenance.

Documentation Technique - Application RAG Multi-Agent

Date: 21 Janvier 2026

Auteur: Analyse Technique Automatisée

Version: 1.0

Table des matières

- 1. [Vue d'ensemble](#vue-densemble)
 - 2. [Architecture Système](#architecture-système)
 - 3. [Analyse Détaillée des Composants](#analyse-détaillée-des-composants)
 - 4. [Flux de Traitement](#flux-de-traitement)
 - 5. [Dépendances et Intégrations](#dépendances-et-intégrations)
 - 6. [Résultats Attendus](#résultats-attendus)
 - 7. [Configuration et Déploiement](#configuration-et-déploiement)
 - 8. [Guide de Maintenance](#guide-de-maintenance)
-

Vue d'ensemble

Description du Projet

L'application RAG Multi-Agent est un système sophistiqué d'Extraction, Traitement et Recherche de Documents basé sur l'architecture LangGraph. Elle est spécialisée dans l'analyse des **Documents Départementaux sur les Risques Majeurs (DDRM)**.

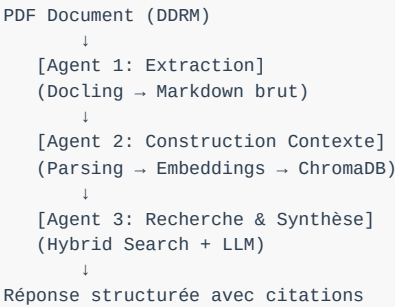
Objectif Principal

Fournir un système complet pour :

- **Extraire** des documents PDF complexes en Markdown structuré
- **Traiter** le contenu pour créer des embeddings vectoriels
- **Rechercher** des informations pertinentes avec une recherche hybride
- **Synthétiser** des réponses intelligentes basées sur des documents

Architecture Générale

L'application suit une architecture **multi-agents orchestrée par LangGraph** :



Principes de Conception

1. **Séparation des responsabilités** : Chaque agent a une fonction spécifique
2. **Modularité** : Composants indépendants et réutilisables
3. **Type Safety** : Utilisation extensive de Pydantic pour la validation
4. **Logging complet** : Traçabilité détaillée avec Loguru
5. **Configuration externalisée** : Settings Pydantic pour tous les paramètres

Architecture Système

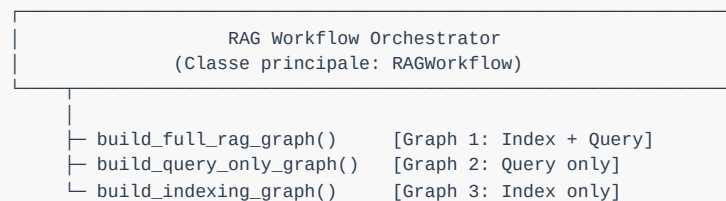
Vue d'ensemble Hiérarchique

```

project/
├── core/                                # Composants partagés
│   ├── base_agent.py                  # Classe abstraite BaseAgent
│   └── models.py                      # Modèles Pydantic (validation type)
├── config/                             # Gestion de configuration
│   └── settings.py                    # Paramètres centralisés (Pydantic)
├── agents/                             # Les trois agents du système
│   ├── agent_1_extraction/            # Extraction PDF → Markdown
│   │   └── pdf_extraction_agent.py
│   ├── agent_2_context/              # Construction contexte
│   │   ├── context_construction_agent.py
│   │   ├── markdown_parser.py
│   │   ├── embedding_service.py
│   │   └── vector_store.py
│   └── agent_3_synthesis/             # Recherche & Synthèse
│       ├── research_synthesis_agent.py
│       ├── hybrid_search.py
│       ├── llm_service.py
│       └── prompts.py
├── workflow/                          # Orchestration LangGraph
│   ├── rag_workflow.py                # Classe orchestratrice principale
│   ├── graph_builder.py              # Constructeur de graphe d'état
│   └── nodes.py                      # Fonctions des nœuds
├── utils/                             # Fonctions utilitaires
│   ├── file_utils.py                 # Gestion de fichiers
│   └── text_utils.py                 # Traitement de texte
├── data/                              # Stockage de données
│   ├── PDFs sources
│   └── chroma_db/                    # Base de données vectorielles
├── output/                            # Fichiers générés
│   ├── markdown/                     # Markdown extraits
│   └── chunks/                       # Chunks JSON
├── main.py                            # Point d'entrée CLI
└── requirements.txt                   # Dépendances Python

```

Flux d'Architecture



Chaque graphe contient des nœuds:

Indexing Graph:

extraction_node → context_construction_node → END

Query Graph:

search_node → synthesis_node → END

Full RAG Graph:

extraction_node → context_construction_node → search_node → synthesis_node → END

Analyse Détaillée des Composants

1. Configuration du Système (config/settings.py)

Objectif

Centraliser tous les paramètres de configuration en utilisant Pydantic pour la validation et le chargement des variables d'environnement.

Paramètres Clés

```
# Chemins du projet
project_root: Path
data_dir: Path = Path("data")
output_dir: Path = Path("output")

# Configuration LLM - Otoroshi
otoroshi_api_url: str
otoroshi_api_key: str

# Embedding
embedding_model: str = "sentence-transformers/all-MiniLM-L6-v2"
embedding_dimension: int = 384

# ChromaDB
chroma_persist_directory: Path = Path("data/chroma_db")
chroma_collection_name: str = "ddrm_documents"

# Chunking
chunk_size: int = 1000
chunk_overlap: int = 200

# Recherche hybride
top_k_results: int = 5
bm25_weight: float = 0.3
vector_weight: float = 0.7
```

Dépendances

- `pydantic.BaseSettings` : Gestion de configuration
- `python-dotenv` : Variables d'environnement

Interactions

- Utilisé par tous les agents et composants
- Singleton pattern via `@lru\_cache`

2. Modèles de Données (core/models.py)

Objectif

Définir les structures de données typées pour l'ensemble du pipeline.

Modèles Principaux

DocumentChunk

```
- id: str (identifiant unique)
- content: str (contenu textuel)
- chunk_type: ChunkType (title, chapter, section, etc.)
- chapter, section, subsection: str (hiérarchie)
- page_number: int
- source_file: str
- embedding: Optional[List[float]]
- metadata: Dict[str, Any]
```

Méthodes:

```
- get_hierarchy_path() → str
- to_context_string() → str (pour LLM)
```

ExtractionResult

```
- source_file: str
- markdown_content: str (contenu brut Docling)
- output_path: str
- success: bool
- error_message: Optional[str]
- processing_time: float
```

SearchResult

```
- chunk: DocumentChunk
- score: float (pertinence)
- rank: int
- source_metadata: Dict[str, Any]
```

SynthesisResult

```
- query: str
- answer: str
- citations: List[Citation]
- sources: List[SearchResult]
- confidence: float
- processing_time: float
```

Enums

- `DocumentType` : DDRM, API, GENERIC
- `ChunkType` : TITLE, CHAPTER, SECTION, PARAGRAPH, TABLE, LIST

Usage dans le Pipeline

- Agent 1 produit → ExtractionResult
- Agent 2 consomme ExtractionResult, produit → DocumentChunk
- Agent 3 consomme DocumentChunk, produit → SynthesisResult

3. Classe de Base pour Agents (core/base_agent.py)

Objectif

Fournir une interface abstraite commune pour tous les agents du système.

Interface

```
class BaseAgent(ABC):
    def __init__(self, name: str, description: str)

    @abstractmethod
    def process(self, input_data: Any) -> Any

    @abstractmethod
    def validate_input(self, input_data: Any) -> bool

    def update_state(self, key: str, value: Any)
    def get_state(self, key: str, default: Any = None) -> Any
    def clear_state() -> None
```

Propriétés Communes

- `name` : Identifiant de l'agent
- `logger` : Logger Loguru avec contexte agent
- `settings` : Configuration (depuis Settings singleton)
- `\_state` : État interne (dictionnaire)

Implémentations

1. `PDFExtractionAgent` (Agent 1)
 2. `ContextConstructionAgent` (Agent 2)
 3. `ResearchSynthesisAgent` (Agent 3)
-

4. Agent 1: Extraction PDF (agents/agent_1_extraction/pdf_extraction_agent.py)

Objectif

Extraire le contenu de fichiers PDF et convertir en Markdown brut en utilisant **UNIQUEMENT Docling**.

Méthode principale : `process()`

Entrées

- Chemin(s) de fichier(s) PDF

Sortie

- Liste de `ExtractionResult` avec contenu Markdown brut

Étapes de Traitement

1. **Validation des fichiers**
2. Vérifie existence des fichiers PDF
3. Contrôle extension `.pdf`

1. **Configuration Docling**

```
PdfPipelineOptions(  
    generate_page_images=False,  
    do_ocr=False,  
    do_picture_classification=False,  
    image_scale=0.3  
)
```

1. **Conversion PDF**

```

converter = DocumentConverter(
    format_options={
        InputFormat.PDF: PdfFormatOption(pipeline_options=...)
    }
)
result = converter.convert(pdf_path)
doc = result.document

```

1. ****Export Markdown****
2. Utilise méthode ``doc.export_to_markdown()``
3. Génère fichier ``.md`` dans ``output/markdown/``

1. ****Gestion des placeholders****
2. ``--- Page Break ---`` pour les sauts de page
3. ``<!__image__>`` pour les images non traitées

Sorties

```

output/markdown/
├── DDRM_vienne_86.md
└── DDRM_deux_sevres.md

```

Configuration

- ``output_dir`` : Répertoire de sortie (default: ``output/markdown``)
- ``PAGE_BREAK_PLACEHOLDER`` : Séparateur de pages
- ``IMAGE_PLACEHOLDER`` : Placeholder pour images

Dépendances

- ``docling>=2.0.0`` : Extraction PDF
- ``loguru`` : Logging

5. Agent 2: Construction du Contexte (agents/agent_2_context/)

Objectif

Parser le Markdown brut, calculer les embeddings, et stocker dans une base de données vectorielle.

5.1 MarkdownParser (markdown_parser.py)

Rôle: Analyser le Markdown structuré et extraire la hiérarchie du document.

Entrée: Contenu Markdown brut (de Docling)

Sortie: `ParsedMarkdown` avec sections organisées

Processus

```
ParsedMarkdown:
- frontmatter: YAML metadata
- title: H1 principal
- sections: List[ParsedSection] (hiérarchie complète)
- links: List[Dict]
- tables: List[str]
```

Méthodes principales

1. ``parse(content: str) → ParsedMarkdown``
2. Extrait frontmatter YAML
3. Détecte titre H1
4. Parse sections avec hiérarchie

1. ``parse_file(path: Path) → ParsedMarkdown``
2. Charge fichier Markdown
3. Applique ``parse()``

1. ``_parse_sections()``
2. Crée arborescence de sections
3. Mappe niveaux Markdown (#, ##, ###) à hiérarchie

Patterns Regex

```
FRONTMATTER_PATTERN = r"^---\s*\n(?:.*)\n---\s*\n"
HEADING_PATTERN = r"^(#{1,6})\s+(.)$"
LINK_PATTERN = r"\[([^\]]+)\]\([([^\)]+)\)"
TABLE_PATTERN = r"(\|.+\|\n)+"
PAGE_COMMENT_PATTERN = r"<!--\s*Page\s+(\d+)\s*-->"
```

5.2 EmbeddingService (embedding_service.py)

Rôle: Calculer les vecteurs d'embeddings pour chunks de texte.

Modèle par défaut: `sentence-transformers/all-MiniLM-L6-v2`
(384 dimensions)

Méthodes principales

1. ``embed_text(text: str) → List[float]``

2. Embedding unique

3. Gère texte vide

1. ``embed_texts(texts: List[str], batch_size=32) → List[List[float]]``

2. Batch processing

3. Barre de progression

4. Filtre textes vides

1. ``embed_query(query: str) → List[float]``

2. Embedding spécialisé pour requêtes

3. Peut ajouter prefix (modèles e5)

Configuration

- Dimension: 384 (all-MiniLM-L6-v2)
- Batch size: 32
- Progress bar: Optionnel

5.3 VectorStoreManager (vector_store.py)

Rôle: Gérer la persistance des embeddings dans ChromaDB ou FAISS.

Approches supportées

- ChromaDB (défaut)
- FAISS (optionnel)

Méthodes principales

1. ``add_chunks(chunks: List[DocumentChunk], embeddings: List[List[float]]) → int``
2. Stocke chunks + embeddings
3. Retourne nombre stocké

1. ``search(query: str, query_embedding: List[float], top_k=5) → List[SearchResult]``
2. Recherche par vecteur
3. Retourne top-k résultats

1. ``get_all_chunks() → List[DocumentChunk]``
2. Récupère tous les chunks indexés

5.4 ContextConstructionAgent (context_construction_agent.py)

Rôle: Orchestrer le pipeline complet : Markdown
→ Embeddings → DB

Flux principal

```
ExtractionResult (Agent 1)
↓
Markdown Parser → ParsedMarkdown
↓
Sections → DocumentChunks
↓
Embedding Service → Vector embeddings
↓
Vector Store → ChromaDB
```

Méthode

principale:

`processmarkdowncontent()`

1. Valide ``ExtractionResult`` liste
2. Pour chaque résultat réussi :
3. Parse Markdown brut
4. Crée chunks sémantiques
5. Calcule embeddings
6. Stocke dans ChromaDB
7. Retourne statistiques

Sortie

```
{
  "status": "success",
  "stats": {
    "documents_processed": int,
    "total_chunks": int,
    "embeddings_computed": int,
    "chunks_stored": int,
    "failed_documents": int
  },
  "chunks": List[DocumentChunk]
}
```

6. Agent 3: Recherche & Synthèse (agents/agent_3_synthesis/)

Objectif

Effectuer une recherche hybride (BM25 + vecteurs) et générer des réponses synthétisées par LLM.

6.1 HybridSearchEngine (hybrid_search.py)

Rôle: Combiner recherche lexicale (BM25) et sémantique (vecteurs).

Architecture

```
Query
↓
BM25 Search → Rankings (0-1)
Vector Search → Similarities (0-1)
↓
RRF (Reciprocal Rank Fusion) → Fusion
↓
Top-K documents
```

Méthodes principales

1. ``index_chunks(chunks: List[DocumentChunk])``
2. Construit index BM25
3. Stocke corpus pour recherche

1. ``bm25_search(query: str, top_k=10) → List[Tuple[int, float]]``
2. Tokenize requête
3. Applique BM25Okapi
4. Retourne (index_chunk, score)

1. ``hybrid_search(query: str, vector_embedding: List[float], top_k=5) → List[SearchResult]``
2. Combine BM25 + Vector
3. Utilise RRF pour fusion
4. Retourne résultats fusionnés

Tokenization

- Lowercase
- Supprime caractères spéci
- Filtre stopwords français
- Supprime tokens courts (<

Stopwords français

le, la, les, de, du, des, u

Poids configurable

- ``bm25_weight`` : Poids BM25 (0-1)
- ``vector_weight`` : Poids similarité (0-1)
- ``rrf_k`` : Constante RRF (défaut 60)

6.2 LLMService (llm_service.py)

Rôle: Interface pour les services LLM (Otoroshi, OpenAI, Mock).

Interface abstraite

```
class LLMService(ABC):
    def generate(prompt: str) -> str
    def chat(messages: List[Dict[str, str]]) -> str
```

Implémentations

1. **OtoroshiLLMService**
2. Via API gateway Otoroshi
3. Endpoint: `/chat/completions`
4. Format OpenAI-compatible
5. Headers: `Authorization: Bearer {api_key}`

1. **OpenAILLMService**
2. Via API OpenAI directe
3. Support de tous les modèles OpenAI

1. **MockLLMService**
2. Pour tests/développement
3. Retourne réponses fixes

Factory

```
def get_llm_service(
    service_type: str,
    api_url: str,
    api_key: str,
    model: str
) -> LLMService
```

6.3 Prompts (prompts.py)

Rôle: Templates de prompts pour la synthèse.

Prompts disponibles

- 1. ****SYNTHESIS_SYSTEM_P**
- 2. Expert DDRM
- 3. Règles de citation
- 4. Base sur passages fournis

- 1. ****SYNTHESIS_PROMF**
- 2. Contexte + question → réponse
- 3. Avec citations

- 1. ****RISK_ANALYSIS**
- 2. Analyse risques spécifique
- 3. Structure : types, zones, mesures

- 1. ****COMPARISC**
- 2. Comparaison entre sources

6.4 ResearchSynt (research_syn

Rôle:
Orchestrer recherche hybride

+
synthèse
LLM.

**Flux
principal**

- Qu
- Va
- Hy
- Se
- LL
- Ex
- Sy

**Méthode
principale:**
process (que
str)
→
SynthesisRes

- 1. **Validation
- 2. Vérifie
que
l'agent
est
prêt
- 3. Vérifie
non-
vide
de
requête

- 1. **Recl
hybrid
- 2. Appell
`hybric
- 3. Retour
top-
K
résulta

- 1. **
pæ
- 2. Fil
le:
pl
pe
- 3. Lir
tai
co
pc
LL

;
;

;
;

Documentation Technique - Application RAG Multi-Agent

Version 1.0 | 21 Janvier 2026 | Analyse Automatisée